TESTING_PLAN

HelixCode Comprehensive Testing Plan

100% Coverage Strategy for Ported Features

Generated: 2025-11-07 **Version**: 1.0 **Target**: 100% test coverage with all tests passing

---

Table of Contents

Executive Summary

Current Status

- **Total Test Files**: 83+
- **Test Categories**: Unit, Integration, E2E, Automation, Load
- **Existing Coverage**: ~70% (estimated)
- **Target Coverage**: 100%

Testing Goals

1. 100% code coverage for all ported features
2. 100% test success rate across all providers

3. Cross-provider compatibility verification
4. Performance benchmarks for all critical paths
5. Integration tests for multi-component workflows
6. E2E tests for complete user scenarios

Test Philosophy

- **Test Early**: Write tests during implementation
- **Test Often**: Automated CI/CD pipeline
- **Test Thoroughly**: Cover happy paths, edge cases, and error scenarios
- **Test Realistically**: Use realistic data and scenarios
- **Test Performance**: Benchmark critical operations

Current Test Status

Existing Test Infrastructure

Unit Tests

Location: `/internal/*/` **Status**: Good Coverage

Package	Coverage	Test Files	Status
`/internal/editor/`	~95%	8 files	Excellent
`/internal/llm/`	~85%	15+ files	Good
`/internal/tools/`	~80%	8 files	Good
`/internal/agent/`	~60%	5 files	Needs improvement
`/internal/workflow/`	~70%	6 files	Needs improvement
`/internal/mcp/`	~50%	2 files	Needs improvement
`/internal/worker/`	~65%	3 files	Needs improvement
`/internal/auth/`	~75%	4 files	Good
`/internal/notification/`	~70%	5 files	Needs improvement
`/internal/repomap/`	~80%	4 files	Good

Integration Tests

Location: `/test/integration/` **Status**: Limited Coverage

Test Suite	Coverage	Status
Database Integration		Good
Slack Integration		Good
Discord Integration		Good
Telegram Integration		Good
Multi-Agent Coordination		Missing
Workflow Integration		Missing
MCP Protocol		Missing

E2E Tests

Location: `/test/e2e/` and `/tests/e2e/` **Status**: Needs Expansion

Test Suite	Coverage	Status
Basic E2E		Good
Anthropic E2E		Good
Gemini E2E		Good
Qwen E2E		Good
Multi-provider Workflows		Missing
Complete Development Cycles		Missing

Automation Tests

Location: `/test/automation/` **Status**: Good Coverage

Test Suite	Status
Anthropic Automation	
Gemini Automation	
XAI Automation	
Qwen Automation	
OpenRouter Automation	
Free Providers Automation	

Load Tests

Location: `/test/load/` **Status**: Limited

Test Suite	Status
Notification Load Test	
Worker Pool Load Test	Missing
LLM Provider Load Test	Missing

Test Types and Coverage Requirements

1. Unit Tests

Target: 100% code coverage per function

Requirements

- Test all public functions
- Test all error paths
- Test boundary conditions
- Test concurrent access where applicable
- Mock external dependencies

Unit Test Structure

```
func TestFeatureName(t *testing.T) {
    tests := []struct {
        name    string
        input   InputType
        want    OutputType
        wantErr bool
    }{
        {"happy path", validInput, expectedOutput, false},
        {"edge case", edgeInput, edgeOutput, false},
        {"error case", invalidInput, nil, true},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := Feature(tt.input)
            if (err != nil) != tt.wantErr {
```

```
                t.Errorf("Feature() error = %v, wantErr %v",
            err, tt.wantErr)
            }
            if !reflect.DeepEqual(got, tt.want) {
                t.Errorf("Feature() = %v, want %v", got, tt.want)
            }
        })
    }
}
```

2. Integration Tests

Target: All component interactions

Requirements

- Test component boundaries
- Test data flow between components
- Test error propagation
- Test transaction boundaries
- Use test databases and mock services

3. End-to-End Tests

Target: Complete user workflows

Requirements

- Test realistic user scenarios
- Test multi-step workflows
- Test error recovery
- Test state persistence
- Use staging environment

4. Performance Tests

Target: All critical operations

Requirements

- Benchmark key operations
- Set performance thresholds
- Test under load
- Test resource consumption
- Profile for bottlenecks

Provider Compatibility Testing

Test Matrix: All Providers × All Features

Provider	Generation	Tools	Vision	Reasoning	Caching	Status
OpenAI						Complete
Anthropic						Complete
Gemini						Complete
AWS Bedrock						Needs reasoning test
Azure OpenAI						Needs caching test
Vertex AI						Needs tests
OpenRouter						Needs tests
Ollama						Partial
Llama.cpp						Basic only
Groq						Basic only
XAI						Needs tests
Qwen						Needs tests
Copilot						Partial

Legend: - Fully tested and working - Partially tested or needs improvement - Not supported or not tested

Provider Test Template

Location: `/internal/llm/providers/*_test.go`

```go
func TestProviderBasicGeneration(t *testing.T) {
    // Test basic text generation
}

func TestProviderToolCalling(t *testing.T) {
    // Test function/tool calling if supported
}

func TestProviderVision(t *testing.T) {
    // Test image analysis if supported
}

func TestProviderReasoning(t *testing.T) {
    // Test extended thinking/reasoning if supported
}

func TestProviderCaching(t *testing.T) {
    // Test prompt caching if supported
}

func TestProviderStreaming(t *testing.T) {
    // Test streaming responses
}

func TestProviderErrorHandling(t *testing.T) {
    // Test error scenarios (rate limits, auth failures, etc.)
}

func TestProviderFallback(t *testing.T) {
    // Test fallback to alternative models
}
```

Cross-Provider Compatibility Tests

Location: `/test/integration/cross_provider_test.go`

Tests to verify: 1. Consistent API across all providers 2. Correct capability detection 3. Automatic fallback behavior 4. Model selection logic 5. Cost estimation accuracy

---

Edit Format Testing

Test Matrix: All Formats × All Scenarios

Format	Simple Edit	Multi-line	Conflict	Rollback	All Languages	Status
Diff						Complete
Whole						Complete
Search/Replace						Complete
Lines						Complete
Udiff						**MISSING**
Diff-fenced						**MISSING**
Editblock						**MISSING**
Editblock-fenced						**MISSING**
Editblock-func						**MISSING**
Editor-diff						**MISSING**
Editor-whole						**MISSING**

Edit Format Test Template

Location: `/internal/editor/*_editor_test.go`

```
func TestEditorSimpleEdit(t *testing.T) {
    // Test single-line edit
}

func TestEditorMultiLineEdit(t *testing.T) {
    // Test multi-line edit
}

func TestEditorConflictHandling(t *testing.T) {
    // Test conflict detection and resolution
}
```

```go
func TestEditorRollback(t *testing.T) {
    // Test rollback on error
}

func TestEditorLanguageSupport(t *testing.T) {
    // Test with Go, Python, JS, TS, Java, C, C++, Rust, Ruby
}

func TestEditorLargeFiles(t *testing.T) {
    // Test with files > 100KB
}

func TestEditorConcurrentEdits(t *testing.T) {
    // Test concurrent edit safety
}

func TestEditorBackupCreation(t *testing.T) {
    // Test automatic backup creation
}
```

Model-Format Compatibility Tests

Location: `/internal/editor/model_formats_test.go`

Tests to verify: 1. Correct format selection per model 2. Format capability checking 3. Automatic format fallback 4. Format complexity scoring 5. Format recommendation accuracy

Current Status: Comprehensive tests exist

Multi-Agent Testing

Test Matrix: Agent Coordination

Test Scenario	Planning	Coding	Testing	Debugging	Review	Status
Single Agent						Partial
Two Agents						Needs tests
All Agents						**MISSING**

Test Scenario	Planning	Coding	Testing	Debugging	Review	Status
Task Delegation						**MISSING**
Result Aggregation						**MISSING**
Conflict Resolution						**MISSING**
Error Recovery						Partial

Required Tests

Location: `/internal/agent/*_test.go`

Planning Agent Tests

```
func TestPlanningAgent_CreatePlan(t *testing.T)
func TestPlanningAgent_BreakdownTasks(t *testing.T)
func TestPlanningAgent_DependencyResolution(t *testing.T)
func TestPlanningAgent_TechnicalSpec(t *testing.T)
```

Coding Agent Tests

```
func TestCodingAgent_Generate(t *testing.T)
func TestCodingAgent_MultiFile(t *testing.T)
func TestCodingAgent_DependencyManagement(t *testing.T)
func TestCodingAgent_BestPractices(t *testing.T)
```

Testing Agent Tests

```
func TestTestingAgent_GenerateTests(t *testing.T)
func TestTestingAgent_ExecuteTests(t *testing.T)
func TestTestingAgent_CoverageAnalysis(t *testing.T)
func TestTestingAgent_IntegrationTests(t *testing.T)
```

Debugging Agent Tests

```
func TestDebuggingAgent_AnalyzeError(t *testing.T)
func TestDebuggingAgent_RootCause(t *testing.T)
func TestDebuggingAgent_GenerateFix(t *testing.T)
func TestDebuggingAgent_RegressionPrevention(t *testing.T)
```

Review Agent Tests

```
func TestReviewAgent_CodeQuality(t *testing.T)
func TestReviewAgent_SecurityAudit(t *testing.T)
func TestReviewAgent_PerformanceAnalysis(t *testing.T)
func TestReviewAgent_BestPractices(t *testing.T)
```

Agent Coordination Tests

Location: `/internal/agent/coordinator_test.go`

```
func TestCoordinator_TaskDelegation(t *testing.T)
func TestCoordinator_ResultAggregation(t *testing.T)
func TestCoordinator_ConflictResolution(t *testing.T)
func TestCoordinator_ErrorPropagation(t *testing.T)
func TestCoordinator_MultiAgentWorkflow(t *testing.T)
```

Workflow Testing

Test Matrix: Autonomy Modes

Mode	Single Task	Multi-Task	Errors	Permissions	Safety Limits	Status
None						Complete
Basic						Needs tests
Basic+						Needs tests
Semi-Auto						Needs tests
Full Auto						Needs tests

Required Tests

Location: `/internal/workflow/autonomy/*_test.go`

Mode Tests

```
func TestModeNone(t *testing.T) {
    // User controls everything
```

```
}

func TestModeBasic(t *testing.T) {
    // Single iteration, no auto-actions
}

func TestModeBasicPlus(t *testing.T) {
    // 5 iterations, no auto-actions
}

func TestModeSemiAuto(t *testing.T) {
    // 10 iterations, auto context gathering
}

func TestModeFullAuto(t *testing.T) {
    // Unlimited iterations, full automation
}
```

Permission Tests

Location: `/internal/workflow/autonomy/permissions_test.go`

```
func TestPermissionEscalation(t *testing.T)
func TestPermissionDenial(t *testing.T)
func TestPermissionTracking(t *testing.T)
func TestPermissionReset(t *testing.T)
```

Safety Tests

Location: `/internal/workflow/autonomy/safety_test.go`

```
func TestMaxIterations(t *testing.T)
func TestMaxRetries(t *testing.T)
func TestContextSizeLimit(t *testing.T)
func TestResourceLimits(t *testing.T)
```

Plan Mode Tests

Location: `/internal/workflow/planmode/*_test.go`

```
func TestPlanCreation(t *testing.T)
func TestOptionGeneration(t *testing.T)
func TestUserSelection(t *testing.T)
func TestProgressTracking(t *testing.T)
func TestModeController(t *testing.T)
```

Snapshot Tests

Location: `/internal/workflow/snapshots/*_test.go`

```go
func TestSnapshotCapture(t *testing.T)
func TestSnapshotRestore(t *testing.T)
func TestSnapshotComparison(t *testing.T)
func TestSnapshotMetadata(t *testing.T)
func TestIncrementalChanges(t *testing.T)
```

Integration Testing

Required Integration Tests

1. Database Integration

Location: `/internal/database/integration_test.go` **Status**: Complete

2. Multi-Agent Coordination

Location: `/test/integration/multi_agent_test.go` **Status**: **MISSING**

```go
func TestMultiAgentWorkflow(t *testing.T) {
    // Plan → Code → Test → Review workflow
}

func TestAgentCommunication(t *testing.T) {
    // Inter-agent message passing
}

func TestAgentStateSync(t *testing.T) {
    // State synchronization across agents
}
```

3. Workflow Integration

Location: `/test/integration/workflow_test.go` **Status**: **MISSING**

```go
func TestPlanToExecution(t *testing.T) {
    // Plan mode → Act mode transition
}

func TestCheckpointRestore(t *testing.T) {
    // Checkpoint creation and restoration
```

```
}

func TestSnapshotWorkflow(t *testing.T) {
    // Snapshot capture and rollback
}
```

4. MCP Protocol Integration

Location: `/test/integration/mcp_test.go` **Status**: **MISSING**

```
func TestMCPToolCalling(t *testing.T) {
    // Tool registration and execution
}

func TestMCPResourceAccess(t *testing.T) {
    // Resource listing and access
}

func TestMCPSession(t *testing.T) {
    // Session lifecycle
}

func TestMCPWebSocket(t *testing.T) {
    // WebSocket transport
}
```

5. Worker Pool Integration

Location: `/test/integration/worker_pool_test.go` **Status**: Partial

```
func TestSSHConnection(t *testing.T)
func TestAutoInstallation(t *testing.T)
func TestHealthMonitoring(t *testing.T)
func TestTaskDistribution(t *testing.T)
func TestWorkerFailover(t *testing.T)
```

6. Notification Integration

Location: `/test/integration/*_integration_test.go` **Status**: Good (Slack, Discord, Telegram)

Need to add:

```
func TestEmailIntegration(t *testing.T)
func TestPagerDutyIntegration(t *testing.T)
```

```go
func TestJiraIntegration(t *testing.T)
func TestMultiChannelBroadcast(t *testing.T)
```

End-to-End Testing

Required E2E Test Scenarios

1. Complete Development Cycle

Location: `/test/e2e/development_cycle_test.go` **Status**: **MISSING**

```go
func TestCompleteDevelopmentCycle(t *testing.T) {
    // 1. Planning phase
    // 2. Code generation
    // 3. Test generation
    // 4. Test execution
    // 5. Debugging failures
    // 6. Code review
    // 7. Deployment
}
```

2. Multi-Provider Workflow

Location: `/test/e2e/multi_provider_test.go` **Status**: **MISSING**

```go
func TestProviderFallback(t *testing.T) {
    // Test automatic fallback when provider fails
}

func TestProviderSwitching(t *testing.T) {
    // Test manual provider switching mid-workflow
}

func TestCostOptimization(t *testing.T) {
    // Test intelligent provider selection for cost
}
```

3. Distributed Workflow

Location: `/test/e2e/distributed_test.go` **Status**: **MISSING**

```go
func TestDistributedTaskExecution(t *testing.T) {
    // Tasks distributed across multiple workers
}
```

```go
func TestWorkerFailureRecovery(t *testing.T) {
    // Worker failure and task redistribution
}

func TestLoadBalancing(t *testing.T) {
    // Task load balancing across workers
}
```

4. Real-World Project Scenarios

Location: `/test/e2e/real_world_test.go` **Status**: MISSING

```go
func TestNewFeatureImplementation(t *testing.T) {
    // Implement a new feature from scratch
}

func TestBugFixWorkflow(t *testing.T) {
    // Identify and fix a bug
}

func TestRefactoringWorkflow(t *testing.T) {
    // Refactor existing code
}

func TestCodeMigration(t *testing.T) {
    // Migrate code between frameworks
}
```

Performance Testing

Benchmarks Required

1. LLM Provider Benchmarks

Location: `/internal/llm/benchmark_test.go`

```go
func BenchmarkProviderGeneration(b *testing.B) {
    // Benchmark text generation across all providers
}

func BenchmarkProviderToolCalling(b *testing.B) {
    // Benchmark tool calling performance
```

```
}

func BenchmarkProviderCaching(b *testing.B) {
    // Benchmark cache hit/miss performance
}
```

2. Edit Format Benchmarks

Location: `/internal/editor/benchmark_test.go`

```
func BenchmarkDiffEditor(b *testing.B)
func BenchmarkWholeEditor(b *testing.B)
func BenchmarkSearchReplaceEditor(b *testing.B)
func BenchmarkLineEditor(b *testing.B)
```

3. Repository Mapping Benchmarks

Location: `/internal/repomap/benchmark_test.go`

```
func BenchmarkTreeSitterParsing(b *testing.B)
func BenchmarkFileRanking(b *testing.B)
func BenchmarkCacheLookup(b *testing.B)
func BenchmarkSymbolExtraction(b *testing.B)
```

4. Multi-Agent Benchmarks

Location: `/internal/agent/benchmark_test.go`

```
func BenchmarkAgentCoordination(b *testing.B)
func BenchmarkTaskDelegation(b *testing.B)
func BenchmarkResultAggregation(b *testing.B)
```

Performance Thresholds

Operation	Target	Threshold
Basic Generation	< 2s	< 5s
Tool Calling	< 3s	< 7s
File Editing (small)	< 100ms	< 500ms
File Editing (large)	< 1s	< 3s
Repository Mapping	< 5s	< 15s
Tree-sitter Parsing	< 100ms/file	< 500ms/file
Agent Coordination	< 500ms	< 2s

Operation	Target	Threshold
Task Distribution	< 200ms	< 1s

Test Execution Plan

Phase 1: Complete Missing Unit Tests (Week 1-2)

Priority: CRITICAL

1. **Agent Tests** (3 days)
    - Complete testing agent tests
    - Complete debugging agent tests
    - Complete review agent tests
    - Add coordinator tests
2. **Workflow Tests** (2 days)
    - Complete autonomy mode tests
    - Add permission tests
    - Add safety limit tests
3. **MCP Tests** (2 days)
    - Complete protocol tests
    - Add transport tests
    - Add session tests
4. **Worker Pool Tests** (2 days)
    - Complete SSH tests
    - Add auto-install tests
    - Add failover tests

Phase 2: Integration Tests (Week 3-4)

Priority: HIGH

1. **Multi-Agent Integration** (3 days)
    - Agent coordination
    - Task delegation
    - Result aggregation
2. **Workflow Integration** (2 days)
    - Plan to execution
    - Checkpoint/restore
    - Snapshot workflows
3. **MCP Integration** (2 days)
    - Tool calling
    - Resource access
    - WebSocket transport

4. **Notification Integration** (2 days)
    - Email integration
    - PagerDuty integration
    - Jira integration
    - Multi-channel broadcast

Phase 3: E2E Tests (Week 5-6)

Priority: HIGH

1. **Development Cycles** (3 days)
    - Complete workflow
    - Multi-provider
    - Distributed execution
2. **Real-World Scenarios** (3 days)
    - New feature implementation
    - Bug fix workflow
    - Refactoring workflow
    - Code migration

Phase 4: Performance Tests (Week 7)

Priority: MEDIUM

1. **Benchmarks** (3 days)
    - LLM provider benchmarks
    - Edit format benchmarks
    - Repository mapping benchmarks
    - Multi-agent benchmarks
2. **Load Tests** (2 days)
    - Worker pool load
    - LLM provider load
    - Concurrent user load

Phase 5: Cross-Provider Compatibility (Week 8)

Priority: HIGH

1. **Provider Matrix** (5 days)
    - Test all 13 providers
    - Test all features per provider
    - Document compatibility matrix
    - Create provider-specific guides

Coverage Metrics

Coverage Goals

Category	Current	Target	Gap
Overall	~70%	100%	30%
Unit Tests	~80%	100%	20%
Integration Tests	~50%	100%	50%
E2E Tests	~40%	100%	60%

Coverage by Package

Package	Current	Target	Priority
`/internal/editor/`	95%	100%	Medium
`/internal/llm/`	85%	100%	High
`/internal/agent/`	60%	100%	Critical
`/internal/workflow/`	70%	100%	Critical
`/internal/mcp/`	50%	100%	High
`/internal/worker/`	65%	100%	High
`/internal/auth/`	75%	100%	Medium
`/internal/notification/`	70%	100%	Medium
`/internal/repomap/`	80%	100%	Medium
`/internal/tools/`	80%	100%	Medium

Coverage Commands

```
# Run tests with coverage
make test-coverage

# Generate coverage report
go test -coverprofile=coverage.out ./...

# View coverage in browser
go tool cover -html=coverage.out

# Coverage by package
go test -cover ./internal/...
```

```
# Detailed coverage
go test -coverprofile=coverage.out -covermode=atomic ./...
```

Test Infrastructure

Required Tools

1. **Testing Framework**: Go testing stdlib + testify
2. **Mocking**: mockery or testify/mock
3. **Coverage**: go tool cover
4. **Benchmarking**: go test -bench
5. **E2E**: Custom test orchestrator (exists at `/tests/e2e/orchestrator/`)
6. **Load Testing**: k6 or vegeta
7. **CI/CD**: GitHub Actions or GitLab CI

Test Data Management

Location: `/test/testdata/`

Required test data: - Sample code files (all supported languages) - Sample LLM responses - Sample git repositories - Sample configuration files - Sample images (for vision tests) - Sample audio (for voice tests)

Mock Services

Location: `/tests/e2e/mocks/`

Existing mocks: - LLM Provider mock - Slack mock

Need to add: - Discord mock - Email mock - PagerDuty mock - Jira mock - SSH server mock - Database mock

---

Test Execution

Local Testing

```
# Run all tests
make test

# Run specific package tests
go test -v ./internal/llm/...
```

```
# Run with coverage
go test -cover ./...

# Run benchmarks
go test -bench=. ./...

# Run integration tests
go test -v ./test/integration/...

# Run E2E tests
go test -v ./test/e2e/...
```

CI/CD Pipeline

GitHub Actions Workflow (`.github/workflows/test.yml`):

```
name: Test Suite

on: [push, pull_request]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-go@v2
        with:
          go-version: 1.24
      - run: make test

  integration-tests:
    runs-on: ubuntu-latest
    needs: unit-tests
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-go@v2
      - run: go test -v ./test/integration/...

  e2e-tests:
    runs-on: ubuntu-latest
    needs: integration-tests
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-go@v2
      - run: go test -v ./test/e2e/...
```

```yaml
  coverage:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-go@v2
      - run: go test -coverprofile=coverage.out ./...
      - uses: codecov/codecov-action@v2
        with:
          files: ./coverage.out
```

Success Criteria

Testing Complete When:

1. All unit tests passing (100% of implemented features)
2. Integration tests covering all component interactions
3. E2E tests covering major user workflows
4. 100% code coverage for ported features
5. All 13 providers tested with all supported features
6. Performance benchmarks within thresholds
7. Load tests passing under expected load
8. CI/CD pipeline green
9. Test documentation complete
10. No flaky tests

Deliverables

1. Test suite with 100% coverage
2. Provider compatibility matrix
3. Performance benchmark report
4. Test documentation
5. CI/CD pipeline configuration
6. Test data repository
7. Mock service implementations

Next Steps

Immediate Actions (Week 1)

1. **Complete agent tests** (3 days)
    - Testing agent: `/internal/agent/types/testing_agent_test.go`

    - Debugging agent: `/internal/agent/types/debugging_agent_test.go`
    - Review agent: `/internal/agent/types/review_agent_test.go`
    - Coordinator: `/internal/agent/coordinator_test.go`
2. **Add workflow tests** (2 days)
    - Autonomy modes: `/internal/workflow/autonomy/*_test.go`
    - Permissions: `/internal/workflow/autonomy/permissions_test.go`
    - Safety limits: `/internal/workflow/autonomy/safety_test.go`
3. **Run current test suite** (1 day)
    - Execute full test suite
    - Generate coverage report
    - Identify gaps
    - Document failures

Medium-term Actions (Weeks 2-4)

1. Complete integration tests
2. Add E2E test scenarios
3. Implement missing mocks
4. Set up CI/CD pipeline

Long-term Actions (Weeks 5-8)

1. Performance benchmarking
2. Load testing
3. Cross-provider compatibility verification
4. Documentation completion

Document Status: COMPLETE **Last Updated**: 2025-11-07 **Next Review**: After Phase 1 completion